

qresearch 使用者指南

给"想用它做研究"的人：怎么跑起来、能得到什么、四个真实案例、怎么读懂输出。
实现原理与全部技术细节见 [manual.md](#)。

1. 这是什么

qresearch 是一个**自主科研闭环系统**：你给它一个凝聚态物理的研究问题（比如"一维 Heisenberg 链的基态能量与 Bethe ansatz 是否一致"），它自动：

1. 理解问题、提出可证伪的假设；
2. 制定数值实验计划 → **交你审批**（这是它唯一必须等你点头的地方）；
3. 执行实验（精确对角化 / DMRG 等，不经 LLM）→ 三层验证（程序/算法/物理）；
4. 分析证据、决定下一步（继续迭代 / 换路线 / 终止 / 宣布结论）；
5. 循环 2-4 直到给出结论，产出**研究报告 + 完整证据链 + 可回放的操作日志**。

多条经验会沉淀到**跨项目记忆库**：上个项目踩过的坑（比如"这个工具要求偶数系统尺寸"），下个项目开始前就会被检索出来提醒，同样的失败不会重演。

你能得到：一份自动生成的 `report.md`、一条每个结论都能回溯到具体实验的证据链、一份逐动作的事件日志、一个越用越准的经验库。**它不做没有来源的断言**——验证不过的实验不能当证据，dry-run 不冒充真计算，宣布结论必须你确认。

2. 安装与准备

环境要求：Windows / git-bash（或相应 shell）、Python ≥ 3.10 、Node.js。

```
# 1) 安装依赖（项目根目录）
uv pip install -e .                # 或 pip install -e .
uv pip install -e ".[ui,viz]"      # 交互会话（rich 进度条）+ 台账可视化

# 2) DSH runtime（首次）：项目内隔离安装，不动全局
# 已随仓库就位则跳过（.dsh-runtime/ 目录存在即可）

# 3) API 凭据（已在环境中则跳过；值不会被打印/记录）
export DEEPSEEK_API_KEY=sk-...

# 4) 连通性自检（真实轮次，~1 分钟）
.venv/Scripts/python.exe -X utf8 examples/dsh_smoke_test.py
```

无需 API key 也能跑：所有 `examples/phase*_demo.py` 不加 `--live` 时使用
离线脚本模型（秒级、确定性），适合先熟悉流程。

3. 快速上手

3.1 三分钟看全流程（离线）

```
.venv/Scripts/python.exe -X utf8 examples/phase5_demo.py
```

离线跑一个 3 轮的完整研究闭环：决策链 `iterate→iterate→declare_result`、
6 条证据、事件日志落盘、报告生成。产物在 `research_data/demo_phase5/`。

3.2 跑你自己的问题（真实 LLM，最小脚本）

```
# my_research.py — 放在项目根目录
from pathlib import Path
from qresearch.core.events import EventLog
from qresearch.core.storage import Storage
from qresearch.dsh_client import DSHClient
from qresearch.memory import MemoryStore
from qresearch.research_loop import run_research_loop

PID = "my_first_run"
DATA = Path("research_data") / PID
DATA.mkdir(parents=True, exist_ok=True)

client = DSHClient(cwd=DATA / "sandbox", dsh_home=DATA / "dsh_home")
storage = Storage(DATA / "state.sqlite")
log = EventLog(DATA / "events.jsonl")
memory = MemoryStore(Path("research_data/memory.sqlite"),
                      markdown_path=Path("research_data/memory.md"))

try:
    summary = run_research_loop(
        client, storage, log, PID,
        "一维自旋 1/2 Heisenberg 链（PBC, J=1）基态能量随 N 的收敛，"
        "并与 Bethe ansatz 热力学极限  $e_0=1/4-\ln 2$  对照",
        rounds=3, auto_approve=True,      # 首次体验可 auto；正式研究请留人工审批
        memory_store=memory,
    )
    print(summary)
finally:
    client.close(); storage.close()

.venv/Scripts/python.exe -X utf8 my_research.py
```

耗时预期：每站点 1–5 分钟（模型 deepseek-v4-flash），3 轮约 20–40 分钟。

去看 `research_data/my_first_run/`：`report.md`（报告）、`events.jsonl`（逐动作日志）、`sandbox/`（站内 agent 的草稿，被隔离在这里）。

建议路径：先用 `auto_approve=True` 跑通 → 正式研究改回 `False`，每轮审批计划（审批动作会以你的身份留痕，而非 system）。

3.3 交互式会话（像 Code Agent 一样，推荐）

安装了 [ui] extra 后，命令行直接敲：

```
qresearch --data-root research_data      # 默认目录可省略
```

进入会话后**用自然语言描述研究问题**（写清模型、观测量、对照判据），确认后自动开跑完整闭环。确认提示符处：

- [y] 以此问题开跑；**回车**取消；**直接输入修正后的问题**则就地替换并重新确认（2026-09-15 实测教训：碎片输入如 y 不会被当成问题开跑，系统会要求重述）；
- 理解站若判定问题**不可解析**（缺少模型/物理量/计算要求），闭环在假设与计划之前直接拦下转人工并列出**需补充的逐项清单**——不会拿着垃圾问题机械往下跑。

运行中你能看到：

- **实时进度**：spinner 显示"当前正在做什么"（哪个站点调用中/第几次尝试、实验完成/失败/验证计数、已用时间），里程碑事件持久打印，绝不刷屏丢信息；
- **计划文档**：每版计划自动写成 <项目目录>/plans/plan_vN.md，审批提示里给出路径——可以用编辑器直接打开改。注意：**绕过审批菜单直接改文件不会生效**，循环会提示"检测到会话外修改"，须按 [e] 走编辑回读（转录→critic→审批）管线才转录成新版计划；
- **轮末节点汇报**：每轮收尾报告决策与证据增量，回车继续 / 输入修改意见 / stop 叫停（运行中随时按一次 Ctrl+C 也会在轮末优雅停，两次立即中断）。

会话内斜杠命令（跨项目管理）：

```
/projects    列出已有项目          /status <pid>    项目进度
/report <pid> 打印研究总结          /plots <pid>     生成三张台账图
/pause <pid>  轮间优雅暂停          /resume <pid>    从台账续跑
/quit        退出
```

会话层的职责边界：它只把你的话路由成**参数/反馈/命令**，零研究决策、零台账写入；所有编辑都要走"转录→语义校验→critic→审批"管线才生效，研究真相仍然只在台账里。

3.4 一行没跑完怎么办？

进程可以中断，状态在库里。重入时改用恢复入口：

```
from qresearch.research_loop import resume_research_loop
summary = resume_research_loop(client, storage, log, PID, rounds=3, ...)
```

计划版本、决策、证据全部从台账衔接（见案例四的实测）。

3.5 交互式研究：在 dsh web 里像带研究生一样驱动（计划 v3）

前面的方式都是“脚本驱动”。v3 提供第三种：DSH 壳作为对话前端，通过 MCP 调用 qresearch 引擎——你说自然语言，agent 调 mcp_qresearch_* 工具，引擎负责一切记账与执行：

```
# 1) 落盘 research profile + skill (幂等; --force 重写 patch)
# skill 装到 <DSH_HOME>/skills/ (DSH 的 skill 发现目录, 源码实证)
.venv/Scripts/python.exe -X utf8 examples/setup_dsh_profile.py --projects-root D:/AI/Agent/

# 2) 核对合并结果 (改→验证→记录), 零警告再启动
# 注意: dsh 必须从仓库根目录启动 (.dsh-runtime 所在地), 换目录会报
# "requires an explicit DSH_HOME"
dsh --profile research --dump-config
dsh --profile research
```

五步走查 (L3): research_open (报项目 id) → understand / hypothesize /
plan_create (返回 job_id, job_status 轮询) → plan_show 看卡片 →

你在真实终端批 qresearch approve <dir> <plan_id>, agent 用 plan_approve
查询确认 → experiments_run → verify → analyze → decide。

两条硬边界 (agent 自己批不了、也宣布不了结论):

- **审批**: plan_approve 是纯查询——只查台账里有没有 actor=HUMAN 的批准事件。写批准只能由你执行: 真实终端 qresearch approve|reject (TTY), 或 qresearch approvals-web --data-root <projects_root> 起一个 localhost 审批页在浏览器里点 (**保真度较低**: 凭据与 agent 同信任域, 台账如实记 channel=webui-local, 报告单列标注; 只监听回环地址)。
- **结论**: decide 的 declare_result/terminate 一律 requires_human=true, 确认经 qresearch conclude <dir> <decision_id> (TTY) 或 approvals-web 页面 (幂等, 报告转 concluded 并标注取得通道)。

其他要点:

- 一切长工具返回 `job_id`，让 agent 用 `job_status / job_result` 轮询；同 key 活跃 job 幂等，重复提交不会加速只会混乱。
- 中途叫停：`job_cancel`。已在跑的实验照常完成，没开工的按取消落账——台账诚实记录，不谎报“已取消”。
- agent 的 bash 现算数字不是实验：要记账走 `adhoc_record`（默认未验证，不得引用为证据）；要可引用就必须用注册工具跑并 `adhoc_verify`。
- `qresearch approvals list <data_root>`：随时只读查看各项目待审批计划。

4. 案例一：Heisenberg 链基态研究（真实 LLM 实测）

这是系统完成的第一个真实研究项目（`research_data/demo_phase8_live/projects/proj_a/`，2 轮、约 20 分钟、151 个事件）：

第 1 轮：模型规划了 8 个偶数尺寸（ $N=6..20$ ）的 ED 单点 + 扫描，共 16 个实验，全部通过三层验证。分析得出：

能量密度按 $1/N^2$ 外推与 Bethe ansatz 热力学极限的相对偏差 **$1.77e-4$**
（含 $1/N^4$ 修正项为 $7.2e-6$ ），远优于预设判据。

决策 iterate——不满足于单点，下一轮换 DMRG 路线扩大尺寸（17 个 dmrg 实验因本机 quimb 缺陷失败，**如实落账**，ED 路线继续）。

第 2 轮结束：模型给出 terminate 建议，rationale 里带着对证据覆盖的自我批评（“本轮可核验证据只覆盖 $N=4$ 与 $N=20$ 两个尺寸，且 $N=4$ 在预注册拟合窗口 $N \geq 8$ 之外”）——不是套话，是对自己结果的审视。

你可以复现：

```
.venv/Scripts/python.exe -X utf8 examples/phase8_demo.py --live
```

（演示跑两个项目：`proj_a` 同款 Heisenberg 研究 + `proj_b` 横场 Ising。）

怎么读结果：打开 `proj_a/report.md` ——假设区每条挂着证伪试验的命运（支持/反驳/未判定），实验表每行是“步骤×工具×验证状态×关键数值”，决策记录的每个 passed 检查项都引用具体 evidence id，而每条 evidence 又指向 `experiments/<exp_id>/result.json`。链条从结论一路闭合到原始数值文件。

5. 案例二：缺一个工具？让它自己造 (tfim_ed)

研究横场 Ising 模型需要 `tfim_ed`，工具库里没有。写一份 Spec
(`tool_specs/tfim_ed.yaml`：目的、输入模型、物理断言、golden 定稿)：

```
.venv/Scripts/python.exe -X utf8 examples/phase6_demo.py --live
```

发生什么：

1. **Spec 先行**：golden 基准的数值来自解析/文献/独立实现，由出题侧锁定；编码站的 prompt 里**没有任何基准数值**（有测试断言这件事）。
2. DSH 编码站交付实现 + 自测脚本（要求自带独立 oracle、声明未用基准常数）。
3. 交付实现跑 golden：首轮 **8/8 通过** (Lanczos/稠密双后端、能隙到 $8.57\text{e-}12$ 量级)。
4. 批评者独立审查：对“能隙何时归零”给出自己的参考值实测复核。
5. 你审批 → 注册 → 可被研究循环使用。审查通过后**晋升**：交付代码逐字节进入 `qresearch/tools/`（只追加注册块），diff 可追溯到 Spec。

值得一提的是插曲：Spec 初版写了一句错误的物理预期（“奇数 N 铁磁有受挫”），**构建者用全枚举 + 解析论证反驳了它**，出题侧核对后采纳修订。“出题人≠答题人”是双向的——出题侧错了也会被纠正。

6. 案例三：经验跨项目复用（不再踩同一个坑）

```
.venv/Scripts/python.exe -X utf8 examples/phase7_demo.py
```

演示里“旧项目”故意安排了一个必然失败的实验（`simple_ed` 被喂了奇数 N ）。项目结束后，这条教训被确定性蒸馏进记忆库的**失败案例层**。随后“新项目”（同问题域的关联函数研究）在制定计划和做决策时，prompt 里被自动注入：

失败案例：`simple_ed` 要求偶数 N （本实验因奇数 N 失败）——
与拟议步骤雷同时，必须先修正做法或明确写出差异。

新项目的计划确实避开了奇数 N ，干净收尾。**机制的可靠性来自确定性**：每条记忆都是代码从台账提取的（哪次实验、什么错误、什么教训），可以一路回放——记忆库里没有“模型总结出来的印象”。

Markdown 镜像（`research_data/memory.md`）供你人工翻阅，四个层各一节。

7. 案例四：把它当研究基础设施来运维

全部演示：`.venv/Scripts/python.exe -X utf8 examples/phase8_demo.py (--live`
则第 5 段接真实 LLM)。五段分别演示：

并行实验：计划里的多个独立实验并发跑（`max_parallel_experiments`），
4 个 0.25s 的实验 0.30s 完成；落账顺序仍然确定（先全 started 再逐个 finished）。

预算闸：`Budget(max_rounds=5, max_experiments=2)`——第 1 轮跑满 2 个实验后，
第 2 轮开始前被代码层拦停：

```
status=budget_exhausted, 实验数预算耗尽 (2/2), rounds_used=1
```

预算是你的执行控制，写在代码层，模型建议不能越过。

暂停/恢复：跑 1 轮 → `touch <项目目录>/PAUSE` → 下一轮边界优雅暂停
（`status=paused`）→ 删除旗标 → `resume_research_loop` 续跑至终止。实测计划
版本 [1,2,3]、决策/证据链全程衔接，进程重启也不丢（状态在 SQLite 里）。

多项目队列：`run_projects` 串行跑一批项目，每项目独立目录 + 独立沙箱
（DSH 站内 agent 的写入被隔离），经验共享进同一个记忆库。

Slurm (HPC)：一行切换执行后端——

```
from qresearch.hpc import SlurmConfig, SlurmRunner
mgr = ExperimentManager(storage, log, runner=SlurmRunner(SlurmConfig(
    partition="gpu", time_min=90, mem_gb=8, cpus=4, account="qm"), dry_run=False))
```

注意诚实边界：本机没有集群时保持默认 `dry_run=True`，它只生成/校验 sbatch
脚本、不提交；dry-run 的结果过不了证据门（不是真实计算就不许当证据）。
`submit_cmd` 支持换成 `ssh user@cluster sbatch` 形态跨机提交。真实提交路径
需在集群上验收。

8. 读懂输出

8.1 run_research_loop 的返回与状态语义

status	含义	你该做什么
terminated	模型判定信息增益耗尽/ 结论成立，建议终止	读报告；terminate 也建议人工复核
budget_exhausted	你设的预算（轮数/实验数/ 墙钟）用完	加预算 resume 续跑， 或就此收尾
paused	PAUSE 旗标生效， 轮间优雅暂停	删旗标后 resume
stopped_by_user	轮末回调返回 "stop" ， 你在中途喊停	resume_research_loop 可随时续跑（台账不丢）
needs_human （含轮次与原因）	校验重试耗尽 / 末轮还想迭代 / runtime 异常	按原因处理： 多半是修正做法后 resume， 或人工接管该轮
report / rounds_used / reason / decisions	summary 附带报告路径、 进度、原因、决策 id 列表	—

8.2 审批触点

需要你的地方只有两类：**每轮计划审批**（plan 通过 critic 之后； auto_approve=True 时代行但以 system 留痕注明"非人工"）和 **declare_result/terminate 确认**（无条件人工，模型层强制，代码不代行）。

计划审批是对话式的（类似 Claude Code 的计划模式）：

```
== 计划待审批 ==（v1，共 8 步）
step_1 [run_experiment] 冒烟测试与约定锁定（工具：tfim_ed）
...
计划文档：research_data/<pid>/plans/plan_v1.md（可直接编辑；e 键走编辑回读流程）
审批：[y]批准 / [c]提修改意见 / [e]编辑文档 / [s]看步骤细节 / [q]放弃：
```

每版计划同时写成 markdown 文档（plans/plan_vN.md），两条修改通道随你选：

- **意见通道 (c / 直接输入文字)**：在审批提示处直接打一段话即可（不必逐条口述），按 c 则可输**多行意见**（空行结束）→ 以 plan_feedback 事件留痕（actor=HUMAN）→ 系统带着你的意见重新制题、重新过 critic、出**新版本计划**再请你批；你的意见在下一版计划 prompt 里是最高优先级，模型不采纳必须写明理由；
- **编辑通道 (e)**：在编辑器里直接改 plan_vN.md（改步骤/工具/输入都行，也可以只在文末"修改意见"节写想法），保存后回终端按回车 → transcribe 站点把你的编辑**转录**回结构化计划（你的话只是"意图的结构化"）→ 语义校验 → critic 独立重审 → 新版本再请你批。**用户编辑不绕过验证管线**；
- **y**：以你的身份（actor=HUMAN）批准落账；
- **s**：展开每步的 inputs/expected_outputs 细节再决定；
- **q**（或输入流结束/EOF/Ctrl+C）：放弃——**绝不默认批准**。

修订次数受上限约束（默认 5 次）防无限对话；每版文档与每条意见都落账可回放。

8.3 中途报告与转向：round_callback

run_research_loop(..., round_callback=...) 给你一个每轮收尾的挂钩，收到 dict 摘要（轮号、本轮决策、本轮实验/证据数、证据总量），返回值决定走向：

返回	行为
None	继续下一轮
"stop"	当轮做完即停（status= stopped_by_user ），部分结果照常出报告，可 resume
"你的修改意见"	意见落账（ user_feedback ，actor=HUMAN）， 注入下一版计划的最高优先级栏，只生效一轮

收束轮（declare_result/terminate）不触发回调——结论走无条件人工确认通道。
GUI/服务程序可借此实现"重要节点推送 + 人随时插话"。

8.4 结果变化可视化

```
from qresearch.viz import plot_project # pip install -e ".[viz]"
plot_project(storage, "my_project", out_dir) # 返回 PNG 路径列表
```

三张图，全部由确定性代码从台账绘制（LLM 不参与，含失败，不做美化）：

1. experiment_tools.png — 工具 × 状态堆叠柱状图（完成/失败分布）；
2. results_vs_param.png — 数值结果随参数的变化，**按计划版本着色**（实心点=验证通过，空心=未通过/未跑），看不同轮次计划产出如何演变；

3. evidence_timeline.png — 证据累计曲线 + 决策时间线（虚线=每轮决策）。

8.5 事件日志 (events.jsonl)

一行一个动作 JSON，按时间序即完整历史。高频事件：

station_call (站点成功) / station_retry (runtime 层失败重试, 含错误) /
approve / experiment_started|finished / verify_experiment / analyze /
decide / memory_retrieved|written / budget_exhausted / paused|resumed /
needs_human / report_generated / station_started (站点开始调用, 进度条数据源) /
plan_feedback (你的审批修改意见, mode=verbal 口述 / edit_doc 文档编辑) /
plan_transcribed (编辑通道转录) / user_feedback (轮末回调意见) /
user_stop (轮末叫停)。调试先看它。

8.6 报告 (report.md)

确定性骨架生成，**置顶是研究总结**：最终状态与决策链 → **结论**（收束决策 rationale 原文，附决策 id）→ 支撑结论的关键证据（逐条挂 evidence id）→ 假设命运（支持/反驳/未判定）→ 规模统计。之后依次：假设（含证伪试验）→ **计划版本历史**（每版的 critic 判定、阻塞项、依据决策、步骤清单——你提过的修改意见体现在版本差异里）→ 实验表（步骤×工具×验证状态×关键结果）→ 分析结论（每条挂实验 id）→ 决策记录（每个 passed 检查项挂 evidence id）→ 局限与不确定性。**对外发布前需人工复核。**

9. 常见问题

Q：站内 agent 在我的仓库里写文件吗？

不会。DSH 站内 agent 的 cwd 是项目沙箱（<项目目录>/sandbox/），草稿都写在那里。直接用 DSHClient() 且不传 cwd 时才指向仓库根——正式使用请总是传沙箱。

Q：站点调用卡住很久？

正常站点 1–5 分钟。默认 15 分钟看门狗会重启 runtime 并自动重试（事件日志里出现 station_retry）；重试耗尽转 needs_human，不会无限挂死。环境很慢可调 QRESEARCH_STATION_TIMEOUT_S。

Q：dmrg 实验大批失败？

本机 quimb 1.15 的已知平台缺陷（局部求值器 NaN），失败已如实落账且

dmrg_adapter 会自动回退 MPO 稠密对角化 ($L \leq 10$ 与 ED 一致到 $1e-15$)。大规模 DMRG 建议换集群 (Slurm 后端)。

Q: 重跑同一项目报 "session already exists"?

DSH session 持久化在 dsh_home。换 project_id, 或复用 resume_research_loop 从台账续跑 (推荐, 状态不丢)。

Q: 项目可以建在仓库目录之外吗?

可以。所有路径都是显式参数: Storage / EventLog / DSHClient(cwd=, dsh_home=) / experiments_root / MemoryStore 想放哪就放哪 (run_projects 的 data_root 同理), 报告自动生成在 state.sqlite 旁边。仓库里强绑定的只有代码与 DSH runtime 二进制 (按包位置解析, 与数据位置无关)。注意**定好位置再跑**: 实验产物在台账里记绝对路径, 跑完再挪目录会导致产物路径失效; DSH session 也按沙箱绝对路径归档。

Q: 怎么加一个新工具?

先写 Spec + golden (tool_specs/, 数值来自解析/文献/独立实现), 要么手写后按 ToolSpec 注册, 要么走 §5 的构建流程。工具被 golden 覆盖后才能产出 passed 证据。

Q: 模型建议的计划不靠谱?

三道防线: PLAN 语义校验 (工具名/动作/scan 格式不对直接打回重写)、critic 独立物理审查 (blocker 强制修订)、你的审批。放开 auto_approve 前先看几轮它规划的质量。

10. 去哪里看更多

- 实现细节与全部机制: [manual.md](#) (技术手册)
- 开发过程与验收记录: [PROGRESS.md](#)
- 原始计划与验收标准: 《基于DSH的量子多体自主科研系统开发计划_v2.md》§9
- 各阶段演示源码: `examples/phase2..8_demo.py` (每份文件头有"它演示什么")